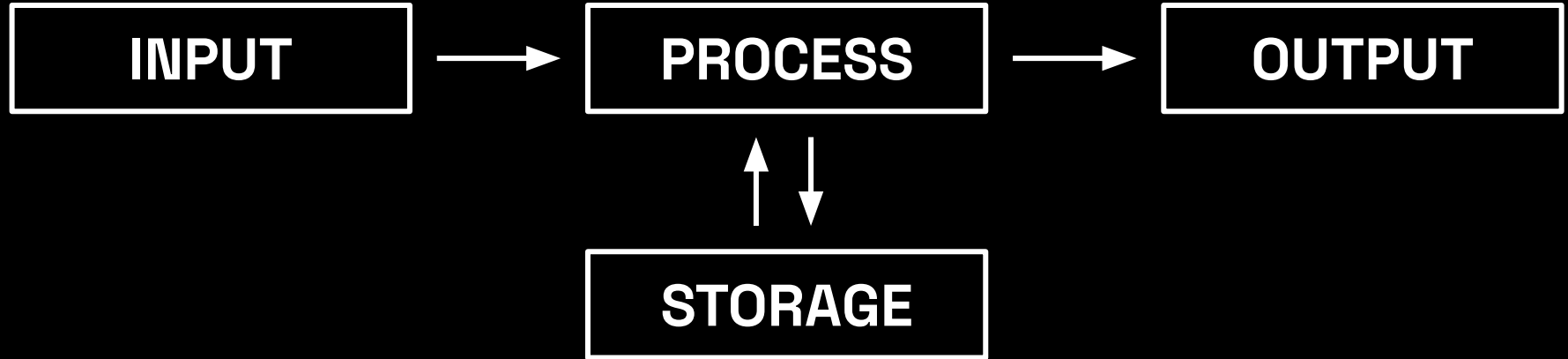




Introduction to
Object-Oriented Programming (OOP)

Programming

Programming



Variables *small pieces of data*

Control flow *if, if else, switch, loops, ...*

Functions *reusable blocks of code*

Data Structures *strings, numbers, arrays, objects*

Procedural Programming

Procedural Programming

What we have done so far!

- Programming paradigm (style of programming)
- Step by step
- Splitting up programs into functions (procedures) and variables (data)

DEMO: 01-procedural-programming



Object-Oriented Programming (OOP)

Object-Oriented Programming (OOP)

- Other programming paradigm (style of programming)
- Modelling a system as **a collection of objects**
- Each object represents some particular aspect of the system
- Objects contain both **methods** (functions) and **properties** (data)
- Fundamental to many programming languages, including PHP, C#, C++

Three main concepts:

1. Classes and Instances
2. Inheritance
3. Encapsulation



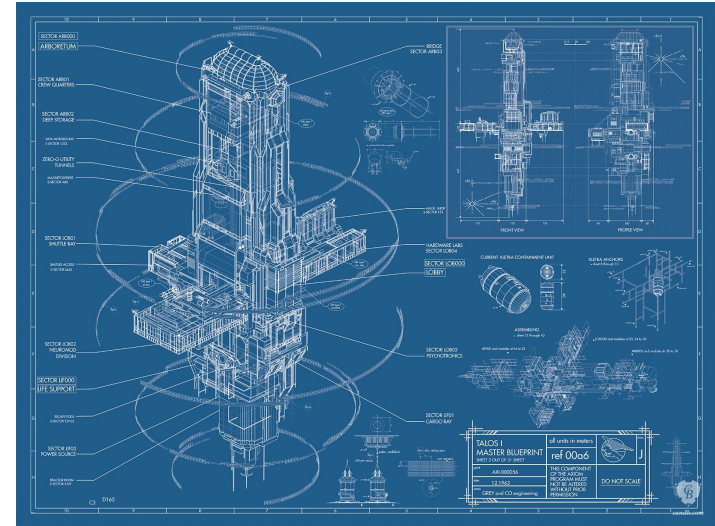
1. Classes and Instances

Class

The blueprint (template) for creating an object

A class describes the:

- **Properties:** data, the characteristics
- **Methods:** behaviour, the functionality



Example: Human Class

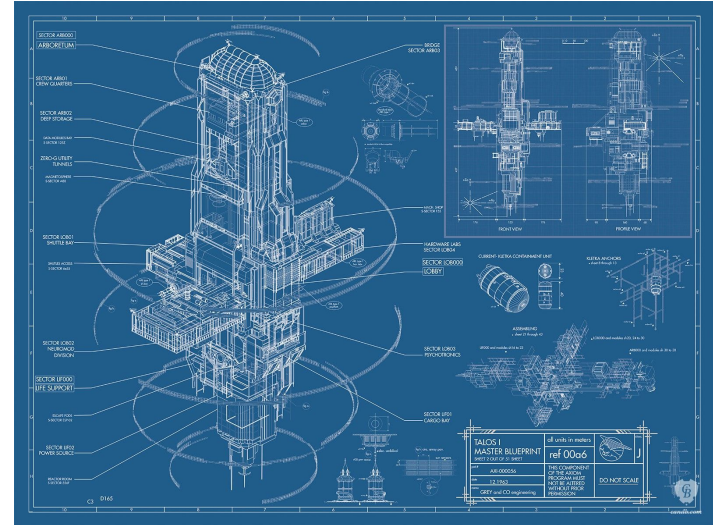
What describes a simple human being?

Properties

- height
- weight
- eye colour
- hair colour
- hobbies
- siblings
- ...

Methods

- sleep
- wake up
- eat
- commute
- study
- exercise
-



Example: Car Class

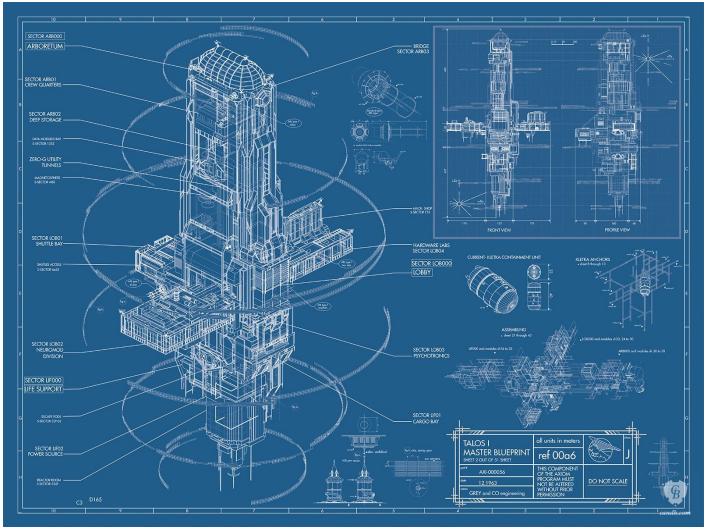
What describes a car?

Properties

...

Methods

...



Example: Car Class

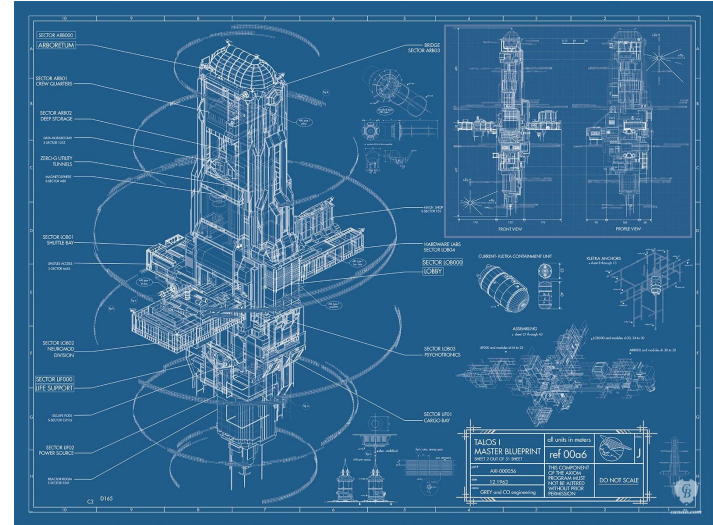
What describes a car?

Properties

- colour
- brand
- price
- horsepower
- ...

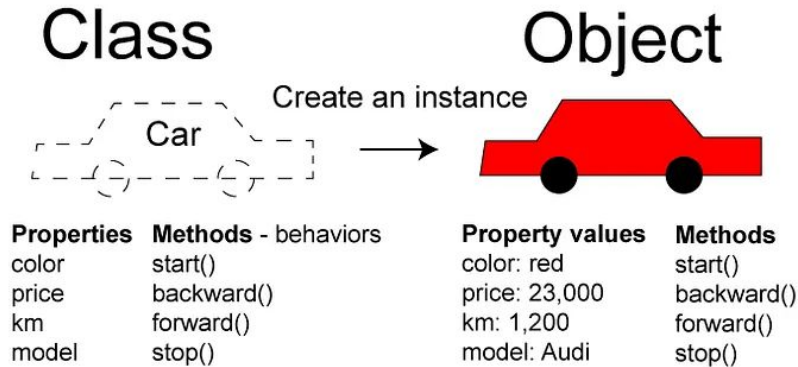
Methods

- drive
- turn
- brake
- park
- ...

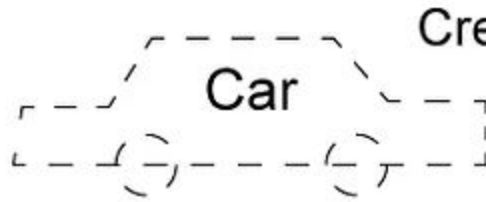


Object

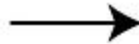
- An instance of a class
- An object is created with the data types and behaviour as described by the class



Class



Create an instance

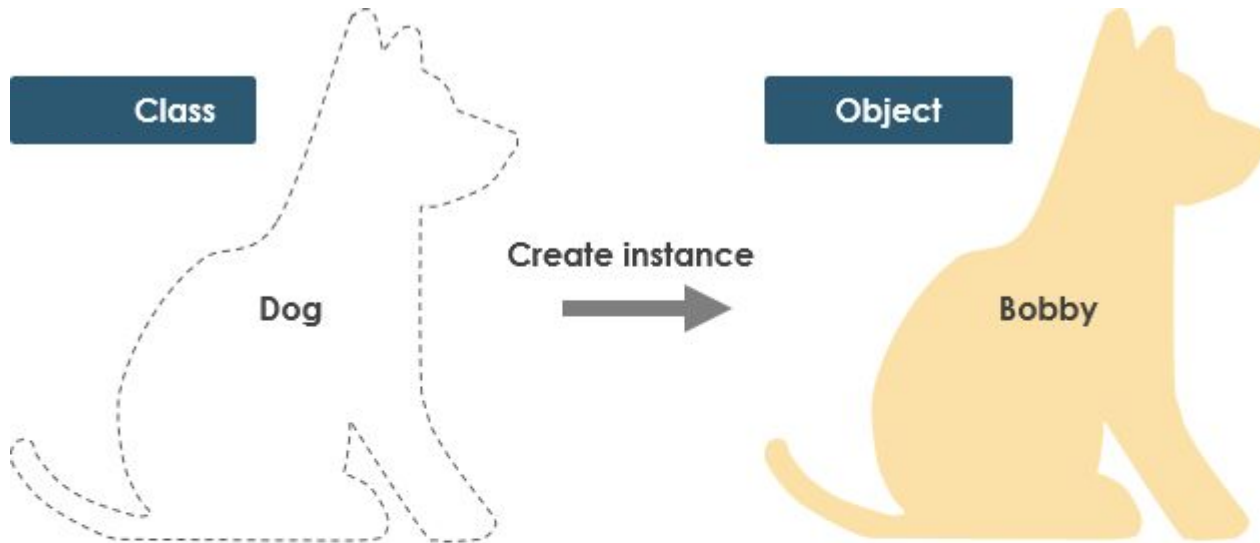


Object



Properties	Methods - behaviors
color	start()
price	backward()
km	forward()
model	stop()

Property values	Methods
color: red	start()
price: 23,000	backward()
km: 1,200	forward()
model: Audi	stop()



Properties

Color

Eye Color

Height

Length

Weight

Methods

Sit

Lay Down

Shake

Come

Property Values

Color: Yellow

Eye Color: Brown

Height: 17 in

Length: 35 in

Weight: 24 pounds

Methods

Sit

Lay Down

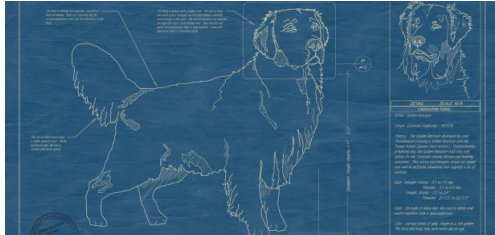
Shake

Come

Class

Blueprint (template) for creating an Object

- Data (properties)
 - colour, age, weight
- Methods (functionality)
 - run, eat, bark, poop



Object

An object is created with the data types and behaviour as described by the class.



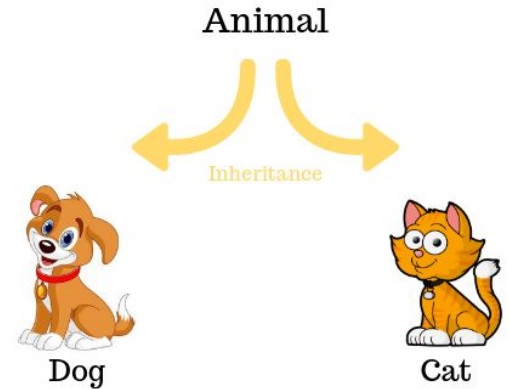
2. Inheritance

Inheritance

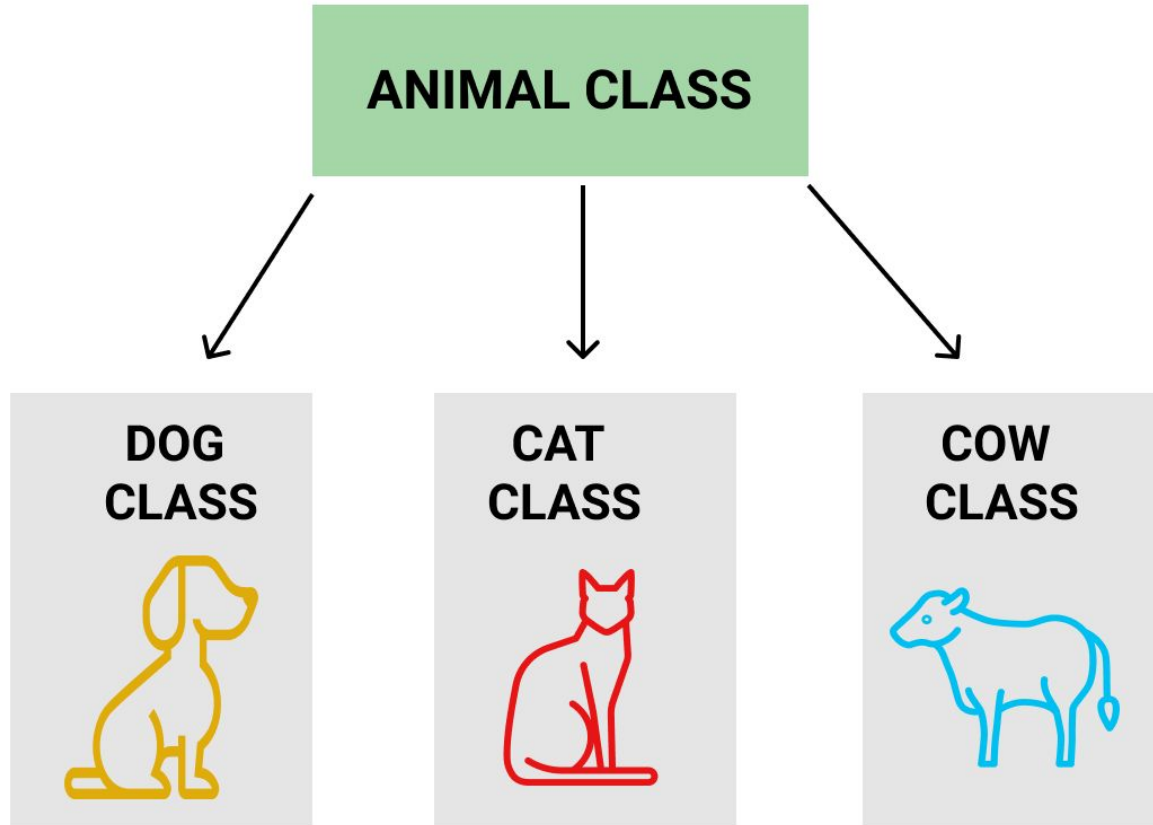
Inheritance allows a class (child/subclass) to inherit properties and behaviours (methods) from another class (parent/superclass)

Why?

- Promotes code reusability
- Creates a hierarchy of classes
- Simplifies maintenance by modifying the parent class without changing child classes



[Source](#)



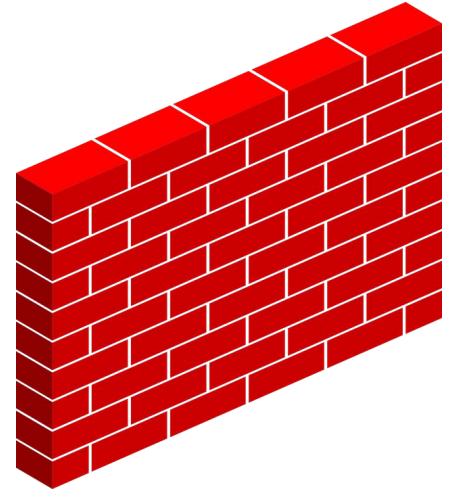
3. Encapsulation

Encapsulation

Restricting access to certain properties and methods of a class

to protect the internal state and enforce controlled interactions

- **Public:** properties and methods accessible to external code (outside the class)
- **Private:** properties and methods that are hidden from external code and only accessible within the class.



OOP in JavaScript

```
class Name {  
  constructor(param1, param2) {  
    this.property1 = param1;  
    this.property2 = param2;  
    this.property3 = 50 // fixed value  
  }  
  method1() {  
    // instructions for behaviour 1  
  }  
}
```



```
class Dog {  
    constructor(age, name) {  
        this.age = age;  
        this.name = name;  
    }  
    bark() {  
        // instructions for barking  
    }  
    walk() {  
        // instructions for walking  
    }  
}
```



Creating a Class

- Using the class keyword
- Using a constructor function
- Best practice: create a new JavaScript file for every class
- Do not forget to import your new file in index.html

DEMO: 02-class



Constructor

constructor is executed when a new instance (object) is created

It receives parameters who will be used to set the properties describing the object

'**this.**' refers to itself, the object that is being created

```
class Name {  
  constructor(param1, param2) {  
    this.property1 = param1;  
    this.property2 = param2;  
    this.property3 = 50 // fixed value  
  }  
  method1() {  
    // instructions for behaviour 1  
  }  
}
```

Default Values

Sometimes we won't receive parameters

In this case, we can add a default value

If we receive a parameter, we will use it. Otherwise, we will use the default value

```
class Name {  
  constructor(param1, param2 = 100) {  
    this.property1 = param1;  
    this.property2 = param2;  
    this.property3 = 50  
  }  
  method1() {  
    // instructions for behaviour 1  
  }  
}
```

Creating an object (= instance of a class)

```
class Car {  
  constructor(x, y) {  
    this.x = x;  
    this.y = y;  
  }  
  
  display() {  
    rect(this.x, this.y, 100, 50);  
  }  
}
```

Car.js

```
let car;  
  
function setup() {  
  createCanvas(400, 400);  
  car = new Car(0, 0);  
}  
  
function draw() {  
  background(220);  
  car.display();  
}
```

script.js



Class keyword

Car.js

```
class Car {  
  constructor(x, y) {  
    this.x = x;  
    this.y = y;  
  }  
  
  display() {  
    rect(this.x, this.y, 100, 50);  
  }  
}
```

script.js

```
let car;  
  
function setup() {  
  createCanvas(400, 400);  
  car = new Car(0, 0);  
}  
  
function draw() {  
  background(220);  
  car.display();  
}
```

Class

class keyword

In a new JS file

- Car.js (uppercase!)
- 1 file = 1 class
- PascalCase
- update index.html!

constructor

- executed when a new instance is created
- set properties using default values or parameters received

methods



Object

new keyword

creates a new instance of the class

can use the functionalities and properties described in the class.

Exercise Time!

01-in-decrease

02-bouncing-square

03-multiple-in-decrease

04-bouncing-balls

05-ripples



Inheritance

The **extends** means that *Rabbit* inherits all the properties and methods of *Animal*

```
//class definition
class Rabbit extends Animal {
  constructor(name, color) {
    super(name);
    this.color = color;
  }
}
```

```
class Animal {
  constructor(name) {
    this.name = name;
  }
  walk() {
    console.log(`${this.name} walking`);
  }
}
```

The **super** methods passed parameters from *Rabbit* to the constructor of *Animal*

Two curved arrows originate from the `super(name);` line in the `Rabbit` class definition and point to the `constructor(name)` in the `Animal` class definition, illustrating the call to the parent class's constructor.

D

```
const whiteRabbit = new Rabbit("Fluffy", "white");
//
whiteRabbit.walk();
```


Static Functions

The **static** keyword defines a static method or field for a class.

Static properties (fields and methods) are defined on the class itself instead of each instance.

```
class Point {  
  constructor(x, y) {  
    this.x = x;  
    this.y = y;  
  }  
  static distance(p1, p2) {  
    return dist(p1.x, p1.y, p2.x, p2.y);  
  }  
}
```

```
let point1;  
let point2;  
  
function setup() {  
  createCanvas(400, 400);  
  point1 = new Point(50, 50);  
  point2 = new Point(300, 300);  
  const distance = Point.distance(point1, point2);  
}
```



Exercise Time!

06-shapes



Extra Resources

Extra Resources

- [Patt Vira - What is Object-Oriented Programming](#)
- [Object-Oriented Programming, Simplified](#)
- [What is inheritance in programming?](#)



More p5.js?

Unfortunately, this was the last class using p5.js

More p5.js?

- [Patt Vira](#)
- [Gorilla Sun](#)
- [The Coding Train](#)
- [Nature Of Code](#)

